

REST ASSURED

Complete Reasoning Guide

WHAT • WHY • HOW • WHEN

for every concept in the SBlePay API Automation Framework

Building Reasoning Ability to Think Clearly

Framework	SBlePay REST Assured Enterprise Framework
Language	Java 17 + Maven
Libraries	REST Assured 5.4 • TestNG 7.9 • Jackson • Lombok
Audience	Beginners who know Java + Selenium + TestNG
Total Concepts	60+ concepts with step-by-step reasoning

TABLE OF CONTENTS

Chapter 1 — REST API Fundamentals — The Foundation

- What is an API
- REST Architecture
- HTTP Methods
- Status Codes
- Headers
- Request Body & Content Types
- Authentication Types

Chapter 2 — REST Assured Library — Core Concepts

- given() / when() / then() Pattern
- RequestSpecification
- ResponseSpecification
- Serialization & Deserialization
- JsonPath Navigation
- Hamcrest Matchers
- Filters & Interceptors
- Logging in REST Assured

Chapter 3 — Framework Architecture — Design Thinking

- Layered Architecture
- Separation of Concerns
- Single Responsibility
- DRY Principle
- Builder Pattern
- Factory Pattern
- Fluent Interface Pattern
- Singleton Pattern

Chapter 4 — Configuration Management

- Why Externalise Config
- Properties Files
- Owner Library
- Environment Switching
- Secrets Management

Chapter 5 — Authentication & Token Management

- Dual Token System

- JWT Tokens Explained
- Token Caching
- Thread Safety
- Auto-Refresh on Expiry
- Token Switching

Chapter 6 — Encryption in API Testing

- Why Encrypt Payloads
- AES/CBC Algorithm
- Key Derivation
- Encrypted Request Wrapper
- Decryption for Validation

Chapter 7 — Data Models — POJOs & Builders

- What are POJOs
- Lombok Annotations
- Builder Pattern Deep Dive
- Request POJOs
- Response POJOs
- Wrapper Pattern

Chapter 8 — Payload Management — 8 Approaches

- Hardcoded String
- HashMap
- JSONObject
- POJO + Builder
- External JSON Templates
- Template Engine
- Dynamic Builder
- Comparison Matrix & Recommendation

Chapter 9 — Service Layer — Business Abstraction

- Why Services
- Service Method Anatomy
- Positive Helpers
- Negative Test Helpers
- Dynamic Data Flow Between Services

Chapter 10 — Response Validation Strategy

- What to Validate
- HTTP vs Business Status

- Field Validation
- JSON Schema Validation
- Regex & Pattern Validation
- ResponseValidator Fluent Chain

Chapter 11 — Assertions — Hard, Soft, AssertJ

- Hard Assert
- TestNG Soft Assert
- AssertJ Fluent Assertions
- AssertJ Soft Assertions
- Custom Reusable Assertions
- When to Use What

Chapter 12 — Test Design & TestNG

- Test Class Structure
- BaseTest Pattern
- Groups & Categories
- Dependencies & Priority
- Data Providers
- Parallel Execution
- Suite XML Configuration

Chapter 13 — Logging & Observability

- Why Logging Matters
- Log4j2 Architecture
- Log Levels
- Sensitive Data Masking
- Correlation IDs
- Request/Response Filters

Chapter 14 — Reporting

- Extent Reports Architecture
- Thread-Safe Reporting
- Attaching Request/Response
- Report Customisation

Chapter 15 — Retry, Resilience & Flaky Handling

- What Makes APIs Flaky
- RetryAnalyzer
- Exponential Backoff
- RetryTransformer (Global Application)

- Rate Limiting & 429
- Circuit Breaker Concept
- Idempotency

Chapter 16 — E2E Business Flow Testing

- Why E2E
- Dynamic Data Passing
- Token Switching Mid-Flow
- Dependency Chaining
- Security Negative Tests

Chapter 17 — CI/CD, Docker & Environments

- Jenkins Pipeline
- GitHub Actions
- Docker Containerisation
- Environment Management
- Secrets in CI

CHAPTER 1

REST API Fundamentals — The Foundation

Before learning REST Assured, you need absolute clarity on what REST APIs are, how they work, and why every concept exists. This chapter builds that foundation with step-by-step reasoning.

1.1 What is an API?

● WHAT is it?

API (Application Programming Interface) is a contract that lets two software systems talk to each other. Think of it as a waiter in a restaurant: you (the client) tell the waiter (API) what you want, the waiter goes to the kitchen (server), and brings back your food (response).

● WHY do we need it?

Without APIs, every application would be an isolated island. Your mobile banking app needs to talk to the bank's server. Paytm needs to talk to SBI's payment gateway. Swiggy needs to talk to restaurant systems. APIs make all of this possible.

● HOW does it work?

A client sends a REQUEST to a server URL (called an endpoint). The request contains: (a) the URL — where to send, (b) the HTTP method — what action to perform, (c) headers — metadata like authentication, (d) body — the actual data. The server processes it and returns a RESPONSE containing: (a) status code — success/failure, (b) headers — metadata, (c) body — the result data.

Step 1:

Client constructs a request with URL + method + headers + body.

Step 2:

Request travels over the network (HTTP/HTTPS protocol) to the server.

Step 3:

Server receives the request, validates it, processes business logic.

Step 4:

Server constructs a response with status code + headers + body.

Step 5:

Response travels back to the client.

Step 6:

Client reads the response and acts on it.

■ THINK ABOUT IT:

Every single test you write in REST Assured follows this exact pattern. You construct a request (given), send it (when), and validate the response (then). Understanding this flow is everything.

Interview Q: "What is an API and how does it work?"

Answer: An API is a contract between systems allowing communication. A client sends an HTTP request to a server endpoint with method, headers, and body. The server processes it and returns a response with status code, headers, and body. In REST, these are stateless — each request is independent.

1.2 REST Architecture

● WHAT is it?

REST (Representational State Transfer) is an architectural style — a set of rules for building APIs. RESTful APIs use HTTP protocol and treat everything as a 'resource' identified by a URL.

● WHY do we need it?

Before REST, we had SOAP (XML-based, heavy, complex WSDL contracts). REST is simpler, lighter, uses JSON (easy to read), and works with any programming language. That's why 95% of modern APIs are REST.

● HOW does it work?

REST has 6 constraints that make it work: (1) Client-Server separation — client and server are independent. (2) Stateless — every request contains ALL information needed, the server remembers nothing between calls. (3) Cacheable — responses can say 'cache me for 1 hour'. (4) Uniform Interface — consistent URL patterns and HTTP methods. (5) Layered System — client doesn't know if it's talking to the server or a load balancer. (6) Code on Demand (optional) — server can send executable code.

■ THINK ABOUT IT:

The STATELESS constraint directly impacts our testing. It means we must send authentication tokens with EVERY request. The server won't remember that we authenticated 2 seconds ago. That's why TokenManager adds the Bearer token to every API call — not just the first one.

1.3 HTTP Methods

● WHAT is it?

HTTP methods (also called verbs) tell the server WHAT ACTION to perform on the resource. The 5 main ones are GET, POST, PUT, PATCH, DELETE.

Method	Action	Has Body?	Idempotent?	SBlePay Example
GET	Read/Retrieve data	No	Yes	GET /customer/{customerId}
POST	Create new resource	Yes	No	POST /customer/create
PUT	Replace entire resource	Yes	Yes	PUT /customer/{id}
PATCH	Update part of resource	Yes	No	PATCH /customer/{id}
DELETE	Remove resource	No	Yes	DELETE /customer/{id}

● WHY do we need it?

Using the correct HTTP method communicates intent clearly. A POST means 'create something new'. A GET means 'give me data without changing anything'. This matters for caching, security, and idempotency.

■ THINK ABOUT IT:

Idempotent means calling it multiple times has the same effect as calling it once. GET is idempotent: reading a customer 10 times doesn't change anything. POST is NOT idempotent: calling customer/create 10 times creates 10 customers. This is why retry logic must be careful with POST endpoints — our DataGenerator creates unique IDs to prevent duplicates on retry.

Interview Q: "What is the difference between PUT and PATCH?"

Answer: PUT replaces the ENTIRE resource (send all fields, even unchanged ones). PATCH updates only the specified fields. PUT is idempotent (call twice, same result). PATCH may not be. In practice, most APIs use POST for updates (like SBlePay) rather than strict PUT/PATCH.

1.4 HTTP Status Codes

● WHAT is it?

A 3-digit number in every HTTP response that tells you whether the request succeeded or failed, and WHY. They are grouped by the first digit.

Range	Category	Meaning	Common Codes
1xx	Informational	Request received, still processing	100 Continue
2xx	Success	Request succeeded	200 OK, 201 Created, 204 No Content
3xx	Redirection	Need to go somewhere else	301 Moved, 302 Found
4xx	Client Error	YOUR request was wrong	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	SERVER failed to process	500 Internal Error, 502 Bad Gateway, 503 Service Unavailable

● WHY do we need it?

Status codes are the FIRST thing you validate in every API test. A 200 means 'proceed to check the body'. A 500 means 'server crashed, don't bother checking body fields'. A 401 means 'your authentication is wrong'.

● HOW does it work?

In REST Assured: `response.getStatusCode()` returns the integer. In our framework: `validate(response).validateStatusCode(200)` is always the first assertion.

■ THINK ABOUT IT:

SBlePay has a quirk: it returns HTTP 200 even for BUSINESS errors. The actual success/failure is in the JSON body: `{"status": 1}` for success and `{"status": 0, "errors": [...]}` for failure. That's why our `ResponseValidator` checks BOTH the HTTP status code AND the business status field. Many beginners only check status code and miss real failures.

Interview Q: "What is the difference between 401 and 403?"

Answer: 401 Unauthorized = 'I don't know who you are' (missing or invalid token). 403 Forbidden = 'I know who you are, but you're not allowed' (valid token but wrong permissions). In SBlePay, using an ACCESS token for the Booking endpoint returns 403 — the token is valid but it's the WRONG TYPE.

1.5 HTTP Headers

● WHAT is it?

Key-value pairs sent with every request and response that carry metadata — information ABOUT the request, not the actual data. Like the envelope of a letter vs the letter inside.

Header	Purpose	SBlePay Usage
Content-Type	Format of the body	application/json (all APIs)
Authorization	Who is calling	Bearer {accessToken} or Bearer {txnToken}
Merchant-API-Key-Id	Merchant identity	Token generation endpoint only
Merchant-API-Key-Secret	Merchant secret	Token generation endpoint only
x-referrer	Origin identifier	Transaction token endpoint only
X-Correlation-ID	Request tracing	Added by our framework for debugging

● WHY do we need it?

Headers separate 'what the data is' from 'the data itself'. Authentication goes in headers (not the body) because it's metadata. Content-Type tells the server how to parse the body. Without it, the server wouldn't know if the body is JSON, XML, or plain text.

● HOW does it work?

In REST Assured: `given().header("Authorization", "Bearer " + token)`. In our framework: `ApiClient` adds common headers automatically. You never set Content-Type manually — the `RequestSpecBuilder` does it once.

1.6 Request Body & Content Types

● WHAT is it?

The actual data payload sent with POST/PUT/PATCH requests. The Content-Type header tells the server how to interpret it. The most common format today is JSON (JavaScript Object Notation).

● WHY do we need it?

APIs need structured data to process. 'Create a customer named John with email john@test.com' needs to be sent in a machine-readable format. JSON is human-readable too, which makes debugging easy.

● HOW does it work?

JSON structure: objects use `{}` with key-value pairs. Arrays use `[]`. Nesting is unlimited. Example: `{"name": "John", "addresses": [{"city": "Mumbai"}, {"city": "Delhi"}]}`

■ THINK ABOUT IT:

In SBlePay, some APIs accept PLAIN JSON while others require ENCRYPTED JSON wrapped in a special structure: `{"encryptRequest": "encrypted_string", "key": "hval"}`. Our Service layer handles this transparently — tests never deal with encryption directly. This is a direct application of the Separation of Concerns principle.

1.7 Authentication Types

● WHAT is it?

How the server verifies WHO is making the request. Without authentication, anyone could access your bank account via the API.

Type	How It Works	Security Level	Used By
API Key	Secret key in header/query param	Low	Simple services, internal APIs
Basic Auth	Username:password Base64-encoded in header	Low-Medium	Internal tools
Bearer Token (JWT)	Token in Authorization header	High	SBlePay, most modern APIs
OAuth 2.0	Token exchange flow with scopes	Very High	Google, Facebook, GitHub
mTLS	Both client+server verify certificates	Highest	Banking, military

● WHY do we need it?

Different APIs need different security levels. A weather API might use just an API key. A payment gateway like SBlePay uses JWT tokens because they are: (a) time-limited (expire after 20 minutes), (b) self-contained (carry role and merchant ID), (c) tamper-proof (signed with a secret).

● HOW does it work?

SBlePay flow: (1) Send Merchant-API-Key-Id + Secret in headers → receive ACCESS JWT. (2) Send ACCESS JWT in Authorization header for customer/order APIs. (3) Send orderHash → receive TRANSACTION JWT + hval. (4) Send TRANSACTION JWT for booking/payment APIs. Two separate tokens for two security domains.

■ THINK ABOUT IT:

Why TWO tokens? Security principle of LEAST PRIVILEGE. The merchant operations token (ACCESS) should never be able to initiate payments. Even if an attacker steals the ACCESS token, they cannot move money. The TRANSACTION token is order-specific and short-lived. Our TokenManager handles this dual-token system automatically.

CHAPTER 2

REST Assured Library — Core Concepts

Now that you understand REST APIs, let's learn REST Assured — the Java library that lets you TEST these APIs. Every concept here is a tool in your toolbox. Knowing WHEN to reach for each tool is what separates a beginner from a professional.

2.1 The given() / when() / then() Pattern

● WHAT is it?

REST Assured's BDD (Behaviour-Driven Development) syntax that divides every API test into three phases: ARRANGE (given), ACT (when), ASSERT (then). This maps directly to the AAA pattern you already know from testing.

● WHY do we need it?

Readability. A test written as `given().header(...).body(...).when().post(...).then().statusCode(200)` reads almost like English. Anyone on the team can understand what it does without knowing Java deeply. This is crucial in large teams where QA leads, developers, and managers read test reports.

● HOW does it work?

`given()` returns a `RequestSpecification` — you chain methods to configure the request. `when()` triggers the HTTP call with the chosen method. `then()` returns a `ValidatableResponse` — you chain assertions. Under the hood: `given()` builds an internal request object, `when().post()` sends it via Apache `HttpClient`, `then()` wraps the response for assertion.

Step 1:

`given()` — Set up everything about the request: base URI, headers, authentication, content type, body, query parameters, path parameters, cookies, multipart data, timeouts.

Step 2:

`when()` — Choose the HTTP method and endpoint: `.post("/path")`, `.get("/path")`, `.put("/path")`, `.delete("/path")`, `.patch("/path")`.

Step 3:

`then()` — Validate the response: `.statusCode(200)`, `.body("field", equalTo("value"))`, `.header("Content-Type", containsString("json"))`, `.time(lessThan(3000L))`.

```
// SBIEPay example: Generate Access Token
given().baseUrl("https://api-sit.sbiepay.sbi")
    .header("Merchant-API-Key-Id", merchantKeyId)
    .header("Merchant-API-Key-Secret", merchantSecret)
    .when().post("/transaction/v1/token/access")
    .then().statusCode(200)
    .body("status", equalTo(1))
    .body("data[0]", notNullValue());
```

■ THINK ABOUT IT:

In our framework, tests DON'T use `given/when/then` directly. Instead, the Service layer wraps it: `customerService.createRandomCustomer()`. The test only calls the service and validates the result. But understanding `given/when/then` is essential because the service methods USE it internally. You need to read and debug them.

Interview Q: "Can you explain the given-when-then pattern in REST Assured?"

Answer: It's BDD syntax mapping to Arrange-Act-Assert. `given()` configures the request (headers, body, auth). `when()` executes the HTTP method. `then()` validates the response. It makes tests self-documenting. In enterprise frameworks, given/when/then is wrapped inside service classes for reusability, but the pattern remains the foundation.

2.2 RequestSpecification

● WHAT is it?

A pre-configured request template object. It stores base URI, default headers, content type, authentication, timeout, filters — everything that's COMMON across multiple API calls. Built using `RequestSpecBuilder`.

● WHY do we need it?

DRY (Don't Repeat Yourself). If 50 tests call SBlePay APIs, all 50 need: base URI, Content-Type: JSON, logging filters, timeouts. Without `RequestSpecification`, you'd copy-paste these 50 times. With it, you configure once and reuse everywhere.

● HOW does it work?

`RequestSpecBuilder` accumulates settings, then `.build()` creates an immutable `RequestSpecification`. You pass it to `given().spec(mySpec)`. In our framework, `ApiClient` creates the base spec in its constructor and reuses it for every HTTP call.

Step 1:

Create a `RequestSpecBuilder`.

Step 2:

Set base URI: `.setBaseUri(config.baseUrl())`.

Step 3:

Set content type: `.setContentType(ContentType.JSON)`.

Step 4:

Add filters: `.addFilter(new RequestLoggingFilter())`.

Step 5:

Build: `.build()` returns an immutable `RequestSpecification`.

Step 6:

Use: `given().spec(baseSpec).header("extra", "value").post(...)`.

■ THINK ABOUT IT:

Notice that you can ADD more configuration on top of a spec. The base spec has Content-Type: JSON and logging. Individual requests add their own Authorization header. This is LAYERING — common stuff in the spec, specific stuff in the test/service.

2.3 ResponseSpecification

● WHAT is it?

The response counterpart of `RequestSpecification`. A pre-configured set of response checks that apply to EVERY response. Built using `ResponseSpecBuilder`.

● WHY do we need it?

Universal validations. EVERY API response from SBlePay should: be JSON, return within 30 seconds. Instead of asserting these in 50 tests, define them once in a `ResponseSpecification`.

● HOW does it work?

`ResponseSpecBuilder` sets expectations, `.build()` creates the spec, then: `then().spec(responseSpec)`. Our framework applies this in `ApiClient`, so every response automatically gets checked for Content-Type and response time.

● WHEN to use it?

Use for universal checks (Content-Type, response time, generic headers). Do NOT use for API-specific checks (status=1, specific field values) — those go in individual tests.

2.4 Serialization & Deserialization

● WHAT is it?

SERIALIZATION: Converting a Java object into JSON string (for sending). DESERIALIZATION: Converting a JSON string back into a Java object (for reading). Jackson library does this automatically when REST Assured detects it on the classpath.

● WHY do we need it?

Type safety. When you construct a request using a POJO (Plain Old Java Object), the compiler catches typos: `request.setCustomerNam()` won't compile because the field is `customerName`. With raw JSON strings, a typo like `"custmerName"` compiles fine but silently fails at runtime. Type safety prevents an entire category of bugs.

● HOW does it work?

REST Assured + Jackson workflow: (1) You create a Java object (e.g., `CustomerCreateRequest`). (2) You pass it to `.body(request)`. (3) REST Assured calls Jackson's `ObjectMapper.writeValueAsString()` to convert it to JSON. (4) The JSON string is sent as the request body. For responses: (1) `response.as(CustomerResponse.class)` calls `ObjectMapper.readValue()` to parse JSON into a Java object.

Step 1:

Create POJO class with fields matching JSON keys: `private String customerName;`

Step 2:

Add Lombok `@Data` (generates getters/setters) and `@Builder` (generates builder pattern).

Step 3:

Build object: `CustomerCreateRequest.builder().customerName("John").build();`

Step 4:

Pass to REST Assured: `given().body(request)` — Jackson auto-serializes to JSON.

Step 5:

Read response: `response.as(CustomerResponse.class)` — Jackson auto-deserializes.

■ THINK ABOUT IT:

Jackson uses field names as JSON keys by default. If the API uses snake_case (`customer_name`) but your Java uses camelCase (`customerName`), add `@JsonProperty("customer_name")` to the field. SBlePay uses camelCase consistently, so we don't need this — but knowing it exists is important.

Interview Q: "What is the difference between serialization and deserialization?"

Answer: Serialization converts a Java object to JSON (or XML) for sending in request bodies. Deserialization converts JSON responses back to Java objects for type-safe field access. Jackson handles both. The key advantage is compile-time safety and IDE autocomplete vs raw string manipulation.

2.5 JsonPath Navigation

● WHAT is it?

A query language for extracting specific values from JSON responses without deserializing the entire response into a POJO. Like CSS selectors for JSON.

● WHY do we need it?

Sometimes you need just ONE value from a complex response (e.g., customerId from a nested data array). Deserializing the entire response to a POJO just for one field is overkill. JsonPath lets you surgically extract what you need.

● HOW does it work?

REST Assured's `jsonPath()` is Groovy-based: `response.jsonPath().getString("path.to.field")`. For SBlePay responses where data is always in `data[0]`: `response.jsonPath().getString("data[0].customerId")`.

Expression	Returns	SBlePay Example
<code>status</code>	Root-level field	1 (success) or 0 (error)
<code>data[0]</code>	First element of data array	The actual response object
<code>data[0].customerId</code>	Nested field in first element	"CUST001"
<code>errors[0].errorCode</code>	First error's code	"2001"
<code>data.size()</code>	Array length	1
<code>data[0].findAll{it.status=='ACTIVE'}</code>	Groovy filter	Filtered list

● WHEN to use it?

Use for quick one-off extractions (getting an ID to pass to the next API). Use POJOs when you need to access many fields from the same response.

2.6 Hamcrest Matchers

● WHAT is it?

A library of matcher methods used inside `.body()` assertions. They are the VOCABULARY of REST Assured assertions. Examples: `equalTo()`, `notNullValue()`, `hasSize()`, `containsString()`.

● WHY do we need it?

Matchers make assertions readable and powerful. Instead of extracting a value, storing it, then asserting — you write `.body("data[0].status", equalTo("ACTIVE"))` in one line. Hamcrest also gives excellent error messages: 'Expected ACTIVE but was INACTIVE'.

Matcher	Checks	Example
<code>equalTo(x)</code>	Exact equality	<code>body("status", equalTo(1))</code>
<code>notNullValue()</code>	Not null	<code>body("data[0].customerId", notNullValue())</code>
<code>nullValue()</code>	Is null	<code>body("data[0].gstNumber", nullValue())</code>
<code>hasSize(n)</code>	Collection size	<code>body("data", hasSize(1))</code>
<code>greaterThan(n)</code>	Numeric >	<code>body("data[0].amount", greaterThan(0))</code>
<code>containsString(s)</code>	Substring	<code>body("errors[0].errorMessage", containsString("required"))</code>
<code>matchesPattern(r)</code>	Regex	<code>body("data[0].email", matchesPattern("^\\w.+@..."))</code>
<code>hasItems(a,b)</code>	Contains items	<code>body("data[0].roles", hasItems("ADMIN"))</code>

■ THINK ABOUT IT:

In our framework, `ResponseValidator` wraps these Hamcrest matchers inside fluent methods. Instead of `.body("status", equalTo(1))`, you write `.validateSuccess()`. This adds another layer of readability and reusability. But under the hood, it's all Hamcrest.

2.7 Filters & Interceptors

● WHAT is it?

Code that runs BEFORE every request is sent and AFTER every response is received. Like middleware in web frameworks. In REST Assured, you implement the `Filter` interface.

● WHY do we need it?

Cross-cutting concerns — things that apply to ALL requests, not just one. Logging is the primary example. You want EVERY API call logged automatically without adding log statements in every test. Filters do this transparently.

● HOW does it work?

A `Filter` receives the `FilterableRequestSpecification` (the outgoing request) and `FilterableResponseSpecification` (the incoming response), plus a `FilterContext` to proceed. You can read/modify the request, call `context.next()` to actually send it, then read/modify the response.

Step 1:

Create a class implementing `io.restassured.filter.Filter`.

Step 2:

In the `filter()` method, read request details (URL, headers, body).

Step 3:

MASK sensitive data (replace card numbers, CVV, tokens with ***).

Step 4:

Log the masked request details.

Step 5:

Call `context.next(request, response)` to proceed with the actual HTTP call.

Step 6:

Read response details (status, body, time).

Step 7:

Log the response details.

Step 8:

Return the response.

■ THINK ABOUT IT:

Our RequestLoggingFilter masks card numbers (shows first 6 + last 4), CVV (completely hidden), and JWT tokens (shows first 10 chars). This is a PRODUCTION REQUIREMENT. Without masking, test logs would contain sensitive payment data — a serious security violation.

Interview Q: "How do you log API requests without exposing sensitive data?"

Answer: Using a custom REST Assured Filter. The filter intercepts every request before sending, applies regex-based masking on card numbers (show BIN+last4), CVVs (replace with ***), and tokens (truncate), then logs the masked version. The actual request goes through unmodified. This happens transparently for all tests.

CHAPTER 3

Framework Architecture — Design Thinking

This chapter explains WHY the framework is structured the way it is. Understanding these design patterns and principles will help you think like an architect, not just a coder. These principles apply to ANY automation framework, not just REST Assured.

3.1 Layered Architecture

● WHAT is it?

Organising code into horizontal layers where each layer has a specific responsibility and only talks to the layer directly below it. Our framework has 4 layers: Test Layer → Service Layer → Core Layer → Support Layer.

● WHY do we need it?

Imagine a change: SBlePay changes the Customer Create endpoint URL. Without layers, you'd search-and-replace across 15 test files. With layers, you change ONE constant in ApiConstants.java and everything works. Layers LOCALISE changes — a change in one layer doesn't ripple through the entire codebase.

● HOW does it work?

Test Layer: Contains ONLY business assertions (customer should be ACTIVE, error code should be 2001). Knows nothing about HTTP. Service Layer: Builds payloads, encrypts, calls APIs. Returns raw Response. Core Layer: ApiClient (REST Assured wrapper), TokenManager, EncryptionUtil. Support Layer: Config, DataGenerator, JsonUtil, ExtentManager.

■ THINK ABOUT IT:

Ask yourself: 'If I change how encryption works, which files should change?' Answer: ONLY EncryptionUtil.java. If changing encryption forces you to modify test files, your architecture is wrong. Each layer protects the layers above it from change.

3.2 Separation of Concerns (SoC)

● WHAT is it?

Each class/method should have ONE reason to exist. CustomerService knows how to interact with Customer APIs. OrderService knows Order APIs. They don't mix. A test class contains business assertions. A service class contains API call logic. They never overlap.

● WHY do we need it?

When a bug appears in customer creation, you know EXACTLY where to look: CustomerService for API-level issues, CustomerTest for assertion issues, CustomerCreateRequest for payload issues. No hunting through a 2000-line god class.

■ THINK ABOUT IT:

Common beginner mistake: putting API calls, payload construction, encryption, logging, and assertions ALL in the test method. This creates a 100-line test method that's impossible to debug. In our framework, a test method is typically 5-10 lines because ALL complexity lives in services.

3.3 Single Responsibility Principle (SRP)

● WHAT is it?

A class should have only ONE reason to change. TokenManager manages tokens — period. It doesn't log, doesn't encrypt, doesn't build payloads. If token logic changes, only TokenManager changes.

● WHY do we need it?

A class with 5 responsibilities changes for 5 different reasons. Each change risks breaking the other 4 things. A class with 1 responsibility is stable, testable, and easy to understand.

Class	Single Responsibility	Does NOT Do
TokenManager	Manage ACCESS + TXN tokens	Encrypt, log, validate
EncryptionUtil	AES encrypt/decrypt	Manage tokens, build payloads
ApiClient	Send HTTP requests	Business logic, test assertions
ResponseValidator	Validate response fields	Send requests, manage state
DataGenerator	Generate random test data	Send requests, validate responses

3.4 DRY — Don't Repeat Yourself

● WHAT is it?

Every piece of knowledge must have a single, unambiguous representation in the codebase. If the same logic appears in two places, extract it into a shared method/class.

● WHY do we need it?

Duplicated code = duplicated bugs. If the encryption algorithm has a bug and it's copy-pasted in 3 places, you need to fix it in 3 places. Miss one and you have inconsistent behaviour. DRY means fix once, fixed everywhere.

● HOW does it work?

In our framework: (a) All endpoints are in ApiConstants — not scattered across tests. (b) All validation logic is in ResponseValidator — not duplicated per test. (c) All data generation is in DataGenerator — not inline Math.random() calls. (d) All encryption is in EncryptionUtil — not copy-pasted in services.

3.5 Builder Pattern

● WHAT is it?

A design pattern for constructing complex objects step-by-step. Instead of a constructor with 15 parameters (which one is phone? which is email?), you use named methods: `.customerName("John").customerMobile("987...").build()`.

● WHY do we need it?

Readability and safety. Compare: `new Customer("John", "987654", "john@test.com", null, "ACTIVE")` vs `CustomerCreateRequest.builder().customerName("John").customerMobile("987654").customerEmail("john@test.com").build()`. The builder is self-documenting. You can't accidentally swap email and phone.

● HOW does it work?

Lombok's `@Builder` annotation generates the builder automatically. You don't write any builder code yourself. Just annotate the class and use: `ClassName.builder().field1(v1).field2(v2).build()`.

■ THINK ABOUT IT:

Builder pattern is especially valuable for NEGATIVE TESTS. You build a valid request, then deliberately omit one field: `CustomerCreateRequest.builder().customerMobile("987").customerEmail("john@test.com").build()` — notice `customerName` is missing. The test then verifies error code 2001.

3.6 Factory Pattern

● WHAT is it?

A pattern where a method/class creates objects for you, hiding the complex creation logic. You ask for a config object, the factory gives it to you. You don't know (or care) how it was created.

● WHY do we need it?

Config creation involves: loading properties files, determining the environment, layering default + env-specific values. This is complex. Tests shouldn't deal with it. `ConfigFactory.getConfig()` hides all of it behind a simple call.

● HOW does it work?

`ConfigFactory` checks if an instance already exists (singleton). If not, it creates one by reading the environment system property, loading the matching properties file, and returning a typed `FrameworkConfig` interface. Subsequent calls return the same cached instance.

3.7 Fluent Interface Pattern

● WHAT is it?

A pattern where methods return 'this' (the same object), enabling method chaining. Each method call reads like a word in a sentence.

● WHY do we need it?

Compare non-fluent: `validator.validateStatusCode(200); validator.validateSuccess(); validator.validateFieldNotNull("id"); validator.assertAll();` versus fluent: `validate(response).validateStatusCode(200).validateSuccess().validateFieldNotNull("id").assertAll();` The fluent version reads like a specification.

● HOW does it work?

Each `validate*()` method in `ResponseValidator` does its check, adds to the soft assertion list, then returns 'this'. This enables the chain. `assertAll()` at the end triggers all collected assertions.

■ THINK ABOUT IT:

REST Assured itself uses the fluent pattern: `given().header().body().when().post().then().statusCode()`. Our `ResponseValidator` follows the same philosophy. Once you understand fluent interfaces, you can READ any REST Assured code instantly.

3.8 Singleton Pattern

● WHAT is it?

A pattern ensuring a class has only ONE instance in the entire application. ConfigFactory, ExtentManager, and TokenManager all use this.

● WHY do we need it?

You don't want 50 tests each creating their own config object (50 file reads), their own Extent report (50 HTML files), or their own access token (50 token API calls). One instance, shared safely.

● HOW does it work?

Private constructor (prevent external creation) + static getInstance() method that creates the instance on first call and returns the cached instance on subsequent calls. Thread safety via synchronized or volatile double-checked locking.

Interview Q: "What is the Singleton pattern and where do you use it in your framework?"

Answer: Singleton ensures one instance per class. We use it in ConfigFactory (one config for all tests), TokenManager (one token cache, avoiding redundant token calls), and ExtentManager (one report across all tests). Thread-safe implementation using volatile + double-checked locking for parallel execution.

Configuration Management

4.1 Why Externalise Configuration

● WHAT is it?

Moving values that change between environments (URLs, credentials, timeouts) OUT of Java code INTO property files. Code stays the same; config changes per environment.

● WHY do we need it?

Your code should NEVER change between SIT, UAT, and Production. The same compiled code runs everywhere. Only the CONFIG changes. This principle ensures: (a) no accidental production modifications, (b) same code tested in SIT is deployed to production, (c) environment switching via a command-line flag, not code changes.

● HOW does it work?

config.properties holds defaults. sit.properties, uat.properties, prod.properties override specific values. The Owner library loads them based on -Denv=sit flag. ConfigFactory.getConfig().baseUrl() returns the correct URL for the active environment.

Step 1:

Developer runs: `mvn test -Denv=sit` — sets system property `env=sit`.

Step 2:

ConfigFactory reads the env system property: 'sit'.

Step 3:

Owner library loads config.properties first (defaults).

Step 4:

Owner then loads sit.properties, OVERRIDING any matching keys.

Step 5:

config.baseUrl() returns `https://api-sit.sbiepay.sbi/` (from sit.properties).

Step 6:

All tests automatically use the SIT base URL. No code changes needed.

4.2 Owner Library

● WHAT is it?

A Java library that maps .properties files to type-safe Java interfaces. Instead of `String value = properties.getProperty("base.url")` (typo-prone, always returns String), you write `config.baseUrl()` (compile-time checked, returns the correct type).

● WHY do we need it?

Raw `Properties.getProperty()` has problems: (a) key typos are silent (returns null), (b) everything is a String (manual parsing for int/boolean), (c) no default value mechanism, (d) no environment layering. Owner solves all of these.

■ THINK ABOUT IT:

Type safety is a theme throughout this framework: POJOs for payloads (not HashMap), Owner for config (not raw Properties), Enums for constants (not magic strings). Every time you replace a String with a typed alternative, you eliminate an entire category of runtime bugs.

4.3 Secrets Management

● WHAT is it?

Keeping sensitive values (API keys, passwords, encryption keys) OUT of source code and property files. They should come from environment variables, CI/CD secret stores, or vaults at runtime.

● WHY do we need it?

If you commit a real API key to Git, anyone with repo access can use it. Even if you delete it later, it's in Git history forever. In payment systems like SBlePay, a leaked merchant secret could allow unauthorized transactions.

● HOW does it work?

In our framework: (a) Property files use placeholders: `merchant.api.key.id=${MERCHANT_API_KEY_ID}`. (b) In CI/CD (Jenkins/GitHub), secrets are stored in the platform's encrypted store. (c) They're injected as environment variables at runtime: `-Dmerchant.api.key.id=$SECRET`. (d) The code never contains real credentials.

Authentication & Token Management

5.1 Dual Token System

● WHAT is it?

SBlePay uses two separate JWT tokens: ACCESS token for merchant operations (customer, order management) and TRANSACTION token for payment operations (booking, payment). Each has different permissions, different lifetimes, and different generation mechanisms.

● WHY do we need it?

Security principle of LEAST PRIVILEGE and SEPARATION OF DUTIES. The merchant management token cannot initiate payments. Even if compromised, the blast radius is limited. The transaction token is generated per-order and is very short-lived, reducing the window of exposure.

● HOW does it work?

ACCESS token: POST /token/access with Merchant-API-Key-Id + Merchant-API-Key-Secret headers. Returns JWT with role=ACCESS. TRANSACTION token: POST /token/txn/{orderHash} with x-referrer header. Returns JWT with role=TRANSACTION plus hval (encryption key for payment payloads).

■ THINK ABOUT IT:

Notice that the TRANSACTION token also returns hval — the encryption key for payment APIs. This key is DIFFERENT from the merchant-level encryption key. This means every order gets its own encryption key. Even if you decrypt one payment, you can't decrypt others. This is defence-in-depth.

5.2 JWT Tokens Explained

● WHAT is it?

JWT (JSON Web Token) is a compact, self-contained token format. It has 3 parts separated by dots: HEADER.PAYLOAD.SIGNATURE. The payload contains claims like role, merchantId, expiry time. The signature prevents tampering.

● WHY do we need it?

JWTs are STATELESS — the server doesn't need to store session data. All the information is IN the token. This scales horizontally: any server instance can validate the token without checking a central database.

● HOW does it work?

Decoding a JWT (base64): Header: {"alg":"HS512"} tells you the signing algorithm. Payload: {"role":["ACCESS"], "mld":"1000003", "exp":1743079368} tells you the role, merchant ID, and expiry timestamp. Signature: HMAC-SHA512 hash ensuring the payload wasn't modified.

5.3 Token Caching in TokenManager

● WHAT is it?

TokenManager stores generated tokens in memory and reuses them until they expire. It doesn't call the token API for every test — it uses the cached token if it's still valid.

● WHY do we need it?

Without caching, a regression suite of 50 tests would make 50 token API calls before even testing anything. That's 50 unnecessary API calls, adding ~50 seconds of network latency. With caching, it's 1 call for the entire suite.

Step 1:

Test calls `TokenManager.getAccessToken()`.

Step 2:

TokenManager checks: is `cachedAccessToken` null OR expired?

Step 3:

If valid: return cached token immediately (no API call).

Step 4:

If expired/null: call `POST /token/access` to generate new token.

Step 5:

Store new token in cache with its expiry timestamp.

Step 6:

Return the fresh token.

5.4 Thread Safety

● WHAT is it?

When tests run in PARALLEL (multiple threads), multiple threads might try to read/write the cached token simultaneously. Without thread safety, two threads could both see an expired token and both try to refresh it, causing a race condition.

● WHY do we need it?

TestNG can run tests in parallel for speed. If TokenManager isn't thread-safe, parallel runs will have intermittent failures — the worst kind of bug because it's not reproducible consistently.

● HOW does it work?

We use synchronized blocks around the token check-and-refresh logic. Only one thread can enter the critical section at a time. The first thread refreshes the token; subsequent threads find the cached token already refreshed and return it immediately.

■ THINK ABOUT IT:

Thread safety is not just for TokenManager. ExtentManager (report writing), ApiClient (if shared), and any shared mutable state need it. Our framework uses the `volatile` keyword for the singleton instances and `synchronized` for the critical sections.

Encryption in API Testing

6.1 Why Encrypt Payloads

● WHAT is it?

Certain SBlePay APIs (Customer Create, Order Create, Payment Initiation) require the request body to be AES-encrypted before sending. The server decrypts it, processes the data, and responds.

● WHY do we need it?

Defence-in-depth. Even though HTTPS already encrypts the transport layer, application-level encryption adds another barrier. If a man-in-the-middle attack compromises TLS (unlikely but possible), the actual business data is still encrypted. For payment data (card numbers, CVV), this is a PCI-DSS requirement.

● HOW does it work?

Algorithm: AES (Advanced Encryption Standard) with CBC (Cipher Block Chaining) mode and PKCS5Padding. The encryption key is derived from the provided key string. An IV (Initialisation Vector) adds randomness so the same plaintext encrypts differently each time.

Step 1:

Service layer builds the plain JSON payload (e.g., customer data).

Step 2:

Service calls `EncryptionUtil.encrypt(plainJson, encryptionKey)`.

Step 3:

`EncryptionUtil` derives a 256-bit AES key from the provided key string.

Step 4:

Generates a random 16-byte IV (Initialisation Vector).

Step 5:

Encrypts the JSON using AES/CBC/PKCS5Padding.

Step 6:

Returns Base64-encoded encrypted string.

Step 7:

Service wraps it: `{"encryptRequest": encryptedString, "key": hvalKey}`.

Step 8:

Sends the wrapped payload to the API.

■ THINK ABOUT IT:

Tests don't know encryption exists. The Service layer handles everything. A test says: `customerService.createRandomCustomer(encKey)` and gets back a `Response`. This is the power of layered architecture — complexity is hidden behind simple interfaces.

Data Models — POJOs & Builders

7.1 What are POJOs

● WHAT is it?

POJO (Plain Old Java Object) is a simple Java class with private fields, getters, setters, and constructors. No framework-specific inheritance. Used to represent data structures — in our case, API request and response bodies.

● WHY do we need it?

POJOs give you: (a) Compile-time type checking — misspell a field name and the compiler catches it. (b) IDE autocomplete — type 'request.' and see all available fields. (c) Refactoring safety — rename a field and the IDE updates all usages. (d) Self-documentation — the class IS the API contract.

7.2 Lombok Annotations

● WHAT is it?

Lombok is a compile-time annotation processor that generates boilerplate code. `@Data` generates getters, setters, `toString()`, `equals()`, `hashCode()`. `@Builder` generates the builder pattern. `@Slf4j` generates a 'log' field for logging.

● WHY do we need it?

Without Lombok, a POJO with 10 fields needs ~100 lines of getters/setters/constructors/builder. With Lombok, it's ~15 lines. Less code = fewer bugs = easier to read.

Annotation	Generates	Used On
<code>@Data</code>	Getters + Setters + <code>toString</code> + <code>equals</code> + <code>hashCode</code>	All POJOs
<code>@Builder</code>	Builder pattern (<code>ClassName.builder().field().build()</code>)	Request POJOs
<code>@NoArgsConstructor</code>	Empty constructor	Response POJOs (Jackson)
<code>@AllArgsConstructor</code>	Constructor with all fields	Used with <code>@Builder</code>
<code>@Slf4j</code>	Private static final <code>Logger log = ...</code>	Service classes, utilities
<code>@JsonIgnoreProperties(ignoreUnknown = true)</code>	Skip unknown JSON fields	Response POJOs

■ THINK ABOUT IT:

Response POJOs need `@NoArgsConstructor` because Jackson creates an empty object first, then calls setters to populate fields. Without a no-args constructor, deserialization fails with 'Cannot construct instance'. Request POJOs use `@Builder` because we construct them manually in code.

CHAPTER 8

Payload Management — 8 Approaches Compared

This is one of the most debated topics in API automation. There are 8 different ways to construct request payloads. Each has tradeoffs. Understanding all 8 lets you make an INFORMED choice rather than blindly following a tutorial.

8.1 Approach 1: Hardcoded JSON String

● WHAT is it?

Writing the JSON directly as a Java String with escape characters.

Advantages: The fastest way to get something working. Zero setup.

Disadvantages: Terrible for maintenance. Escape characters make it unreadable. No IDE help. Typos are invisible. A single missing quote breaks everything silently.

● WHEN to use it?

Never in production. Only for 30-second debugging.

```
String body = "{\"name\":\"John\",\"email\":\"j@t.com\"}";
```

8.2 Approach 2: HashMap

● WHAT is it?

Using a Map to build key-value pairs, then passing it to REST Assured.

Advantages: Easy to understand. Dynamic values are simple to add. No POJO classes needed.

Disadvantages: No compile-time key validation. Put 'custmerName' instead of 'customerName' and it compiles fine but fails at runtime. No IDE autocomplete for keys.

● WHEN to use it?

Prototyping, quick tests, when the API contract changes frequently.

```
Map<String, Object> body = new HashMap<>(); body.put("customerName", "John");
```

8.3 Approach 3: JSONObject (org.json)

● WHAT is it?

Using the JSONObject class to build JSON programmatically.

Advantages: Supports nesting naturally. Can parse existing JSON strings.

Disadvantages: Same problems as HashMap: no compile-time safety. Extra dependency (org.json). More verbose than HashMap.

● WHEN to use it?

When you need heavy JSON manipulation (merge, transform, query).

```
JSONObject body = new JSONObject(); body.put("customerName", "John");
```

8.4 Approach 4: POJO + Builder (OUR PRIMARY)

● WHAT is it?

Java classes with fields matching JSON keys. Lombok @Builder for fluent construction.

Advantages: Compile-time type safety. IDE autocomplete. Refactoring support. Self-documenting. This is the INDUSTRY STANDARD for enterprise frameworks.

Disadvantages: Requires creating a class per request type. Overhead for one-off tests.

● WHEN to use it?

ALWAYS in production frameworks. Every major company uses this.

```
CustomerCreateRequest.builder() .customerName("John") .customerMobile("9876543210")  
.build();
```

8.5 Approach 5: External JSON Templates (OUR SECONDARY)

● WHAT is it?

JSON files with \${placeholder} tokens stored in src/main/resources. Values injected at runtime.

Advantages: Non-developers can modify templates. Clean separation of data from code. Great for data-driven tests.

Disadvantages: No compile-time validation of placeholders. Forgotten placeholders cause silent failures.

● WHEN to use it?

Data-driven tests, when business analysts maintain test data, API contract documentation.

```
// customer_create.json: {"customerName":"${customerName}","mobile":"${mobile}"}
```

8.6 Approach 6: Template Engine (Mustache/Handlebars)

● WHAT is it?

Full template engine for conditional logic inside JSON templates.

Advantages: Powerful: if/else, loops, partials inside templates.

Disadvantages: Overkill for most API testing. Extra learning curve. Debugging template logic is painful.

● WHEN to use it?

Only when you have genuinely complex conditional payloads.

```
{{#if premium}}"tier":"gold"{{else}}"tier":"basic"{{/if}}
```

8.7 Approach 7: Dynamic Payload Builder

● WHAT is it?

Custom fluent builder that manipulates JSON nodes directly for negative test scenarios.

Advantages: Perfect for negative tests: remove fields, set null, inject invalid values. Dynamic and flexible.

Disadvantages: More complex to implement. Requires Jackson/GSON tree model knowledge.

● WHEN to use it?

Negative tests where you need to corrupt specific fields from a valid baseline.

```
JsonUtil.removeField(template, "customerName"); JsonUtil.setFieldValue(template, "email", "");
```

8.8 Comparison Matrix & Recommendation

Approach	Type Safety	Readability	Maintenance	Best For
1. Hardcoded String	None	Poor	Poor	Never use
2. HashMap	None	Medium	Medium	Quick prototypes
3. JSONObject	None	Medium	Medium	JSON manipulation
4. POJO+Builder	Full	Excellent	Excellent	Production (PRIMARY)
5. JSON Templates	None	Good	Good	Data-driven (SECONDARY)
6. Template Engine	None	Medium	Medium	Complex conditionals
7. Dynamic Builder	Partial	Good	Good	Negative tests

■ THINK ABOUT IT:

The winning strategy is a COMBINATION: POJO+Builder for all happy-path tests (compile-time safety), JSON Templates for data-driven variations (easy to add test data), Dynamic Builder for negative tests (field removal/corruption). This is exactly what our framework implements.

Service Layer — Business Abstraction

9.1 Why a Service Layer

● WHAT is it?

A Java class that encapsulates ALL logic for interacting with a specific API. `CustomerService` knows how to create customers, retrieve them, update status. It handles payload construction, encryption, authentication, and HTTP calls. Tests call ONE method and get back a `Response`.

● WHY do we need it?

Without services, every test contains: build POJO, serialize to JSON, encrypt, wrap in `EncryptedRequestWrapper`, get access token, add to header, call API. That's 15+ lines of infrastructure code per test, repeated across all tests. With services, it's ONE line: `customerService.createRandomCustomer(key)`.

● HOW does it work?

Service method anatomy: (1) Accept parameters (encryption key, specific field overrides). (2) Build the request POJO using `DataGenerator` for random data or specific values. (3) Serialize to JSON. (4) Encrypt if required. (5) Get the appropriate token from `TokenManager`. (6) Call `ApiClient` with the endpoint, body, and auth header. (7) Return the raw `Response` for the test to validate.

■ THINK ABOUT IT:

The Service layer is the BRIDGE between business-language tests and technical HTTP calls. A test says 'create a customer with invalid email' in one line. The service translates that to HTTP mechanics. This is Separation of Concerns in action.

9.2 Negative Test Helpers

● WHAT is it?

Service methods specifically designed for negative testing: `createCustomerMissingField(fieldName)`, `createCustomerWithField(fieldName, invalidValue)`. They build a valid request, then deliberately corrupt one field.

● WHY do we need it?

Negative tests verify that the API correctly REJECTS bad data. Without helpers, each negative test would build its own corrupted payload — lots of duplication. Helpers centralise the corruption logic.

● HOW does it work?

The helper builds a valid request POJO, serializes to JSON (as a Jackson `ObjectNode`), removes/modifies the target field, re-serializes, encrypts, and sends. The test only specifies WHICH field to corrupt and WHAT error code to expect.

Response Validation Strategy

10.1 The Validation Pyramid

● WHAT is it?

A layered approach to validation, from broadest to most specific: HTTP Status → Business Status → Response Time → Required Fields → Field Values → Data Types → Patterns → Business Rules → Schema.

● WHY do we need it?

Validating only the status code catches ~20% of bugs. Adding business status catches ~50%. Adding field validation catches ~80%. Adding schema + patterns catches ~95%. Each layer adds protection. The pyramid ensures you validate at every level.

Step 1:

HTTP Status Code (200, 400, 401, 403, 500) — Did the server even process it?

Step 2:

Business Status (1=success, 0=error) — Did the business logic succeed?

Step 3:

Response Time (< 3000ms) — Is the API within SLA?

Step 4:

Required Fields Not Null — Are all expected fields present?

Step 5:

Field Value Equality — Does status='ACTIVE'? Does currency='INR'?

Step 6:

Data Type Validation — Is amount a number? Is email a string?

Step 7:

Pattern/Regex — Does email match `^[w]+@[w]+\.[a-z]+$`?

Step 8:

Business Rules — Is amount > 0? Is expiry date in the future?

Step 9:

JSON Schema — Does the entire response match the structural contract?

10.2 Soft Assertions in Validation

● WHAT is it?

Our ResponseValidator uses SOFT assertions internally. This means ALL checks run, even if early ones fail. You get a complete report of all failures in one test run.

● WHY do we need it?

With HARD assertions, if the business status check fails, you never learn whether the customerId was null, the email was wrong, or the response time was too slow. With soft assertions, you see ALL 5 failures at once. This dramatically speeds up debugging.

■ THINK ABOUT IT:

The assertAll() call at the end is CRITICAL. Forgetting it means no assertions are actually checked — the test always passes. This is the #1 mistake with soft assertions. Our ResponseValidator always ends with assertAll().

CHAPTER 11

Assertions Deep Dive

11.1 Hard Assert vs Soft Assert vs AssertJ

Type	On Failure	Use Case	Framework
Hard Assert	Stops immediately	When remaining checks are meaningless	TestNG Assert
Soft Assert	Continues, reports all at end	Multiple independent checks	TestNG SoftAssert
AssertJ Hard	Stops immediately + better messages	Single critical checks	AssertJ <code>assertThat()</code>
AssertJ Soft	Continues + fluent + best messages	Multiple checks (OUR CHOICE)	AssertJ <code>SoftAssertions</code>

● WHY do we need it?

We chose AssertJ Soft Assertions because: (a) Fluent syntax: `assertThat(x).isNotNull().isNotEmpty()`. (b) Rich matchers: `.startsWith()`, `.containsPattern()`, `.isCloseTo()`. (c) Descriptive `.as()` labels: `assertThat(id).as('Customer ID should be UUID')`. (d) Excellent failure messages showing expected vs actual with context.

■ THINK ABOUT IT:

Hard assertions are still useful! In E2E tests, if Step 1 (Generate Token) fails, there's no point running Steps 2-7. TestNG's `dependsOnMethods` handles this: if `step01` fails, steps 02-07 are SKIPPED, not failed. This is hard-assert-like behaviour at the test level.

Test Design & TestNG

12.1 BaseTest Pattern

● WHAT is it?

An abstract parent class that ALL test classes extend. Contains shared setup (initialise services, config), teardown (flush reports), and utility methods (validate(), logToReport()).

● WHY do we need it?

DRY for test infrastructure. Every test needs ApiClient, CustomerService, ResponseValidator. Instead of initialising these in 4 test classes, initialise once in BaseTest. All child classes inherit it.

● HOW does it work?

@BeforeSuite (runs ONCE before all tests): create ApiClient and service instances. @AfterSuite (runs ONCE after all tests): flush Extent Reports. Protected helper methods: validate(response) creates a new ResponseValidator, logToReport() attaches request/response to the report.

12.2 Groups & Categories

● WHAT is it?

TestNG groups let you TAG tests with categories: @Test(groups = {"regression", "negative"}). Suite XML can INCLUDE or EXCLUDE groups.

● WHY do we need it?

Run only smoke tests in 2 minutes. Run full regression in 15 minutes. Run only negative tests to verify error handling. Groups give you flexible test selection without maintaining separate test lists.

12.3 Data Providers for Data-Driven Tests

● WHAT is it?

A TestNG feature that runs the same test method multiple times with different data. @DataProvider returns an Object[][] array, and each row becomes one test invocation.

● WHY do we need it?

Instead of writing 6 separate test methods for 6 invalid email formats, write ONE method with a @DataProvider that supplies 6 different emails. Same logic, different data, one method.

Logging & Observability

13.1 Log Levels — Reasoning for Each

Level	Reasoning: Use When...	Framework Example
ERROR	Something FAILED that should not have	API call returned 500, encryption failed
WARN	Something unexpected but recoverable	Token expired, retrying; field was null but optional
INFO	Significant business events	Customer created: CUST001, E2E step passed
DEBUG	Developer details for troubleshooting	Building request payload, decrypting response
TRACE	Ultra-detailed internal state	Raw byte arrays, HTTP wire content

13.2 Correlation IDs

● WHAT is it?

A unique UUID attached to every API request in the X-Correlation-ID header. When a test fails, you give this UUID to the backend team and they can find the exact request in server logs.

● WHY do we need it?

In shared test environments (SIT/UAT), hundreds of requests fly every hour. Without correlation IDs, finding YOUR failed request in the server logs is like finding a needle in a haystack. The correlation ID is the needle's GPS coordinates.

Reporting

14.1 Extent Reports Architecture

● WHAT is it?

Extent Reports is a library that generates beautiful, interactive HTML test reports with dashboards, charts, and detailed test steps. ExtentManager in our framework manages the report lifecycle.

● WHY do we need it?

TestNG's default report is plain XML/HTML with no filtering, no charts, no request/response details. Extent Reports provides: (a) pie charts for pass/fail ratios, (b) filtering by category/author, (c) expandable test steps with attached request/response, (d) environment information, (e) timestamps and duration.

Step 1:

ExtentManager.getInstance() creates the report file (singleton).

Step 2:

TestListener.onTestStart() creates an ExtentTest node for each test method.

Step 3:

During test execution, logToReport() attaches request URL, headers, body, response body, status code.

Step 4:

TestListener.onTestSuccess/Failure() marks the node green/red.

Step 5:

TestListener.onFinish() calls extent.flush() to write the HTML file.

Step 6:

Open target/reports/extent-report.html in a browser.

Retry, Resilience & Flaky Handling

15.1 What Makes APIs Flaky

Cause	Symptom	Solution
Server overload	503 Service Unavailable	Retry with backoff
Network hiccup	Connection timeout	Retry with longer timeout
Rate limiting	429 Too Many Requests	Respect Retry-After header
Shared test env instability	Random 500 errors	Retry 2x, then fail
Slow upstream service	504 Gateway Timeout	Increase read timeout
Data cleanup issues	409 Conflict / 2004 Already Exists	Unique test data per run

15.2 Exponential Backoff

● WHAT is it?

Instead of retrying immediately (which hammers the already-struggling server), you wait progressively longer: 2 seconds, then 4 seconds, then 8 seconds. The wait time **DOUBLES** each retry.

● WHY do we need it?

Immediate retries can make the problem **WORSE**. If the server is overloaded and 100 tests all retry instantly, that's 100 extra requests hitting an already-struggling server. Exponential backoff gives the server breathing room to recover.

Step 1:

Test fails on first attempt.

Step 2:

RetryAnalyzer calculates wait: $2^1 \times 1000 = 2000\text{ms}$ (2 seconds).

Step 3:

`Thread.sleep(2000)`. Then retry.

Step 4:

If still fails: $2^2 \times 1000 = 4000\text{ms}$ (4 seconds). Then retry.

Step 5:

If still fails after maxRetry attempts: mark as genuinely FAILED.

15.3 Global Retry via RetryTransformer

● WHAT is it?

An IAnnotationTransformer that automatically attaches the RetryAnalyzer to EVERY test method. You don't need `@Test(retryAnalyzer = RetryAnalyzer.class)` on each test.

● WHY do we need it?

Adding the retry annotation to 50 test methods is tedious and error-prone (forget one and it has no retry). The transformer applies it globally via the TestNG suite listener. Zero-touch retry for all tests.

CHAPTER 16

E2E Business Flow Testing

16.1 Why E2E Tests

● WHAT is it?

End-to-End tests execute the complete business flow from start to finish: Token → Customer → Order → Status Update → TXN Token → Booking → Payment. Each step uses data from the previous step.

● WHY do we need it?

Unit tests verify individual APIs work. E2E tests verify they work TOGETHER. The biggest bugs in payment systems are INTEGRATION bugs: 'Customer Create returns a customerId that Order Create doesn't accept.' E2E catches these.

16.2 Dynamic Data Passing

● WHAT is it?

E2E tests store outputs from each step (customerId, orderHash, txnToken) in instance variables and pass them to subsequent steps automatically.

Step 1:

step01_GenerateAccessToken() → TokenManager caches the token.

Step 2:

step02_CreateCustomer() → stores customerId in instance variable.

Step 3:

step03_CreateOrder() → uses customerId, stores orderHash + orderRefNumber.

Step 4:

step04_UpdateOrderStatus() → uses orderRefNumber + sbiOrderRefNumber.

Step 5:

step05_GenerateTransactionToken() → uses orderHash, TokenManager caches txnToken.

Step 6:

step06_BookTransaction() → uses orderHash, TokenManager provides txnToken.

Step 7:

step07_InitiatePayment() → uses orderHash, txnToken, hval for encryption.

■ THINK ABOUT IT:

TestNG's `dependsOnMethods` is ESSENTIAL here. If Step 3 (Create Order) fails, Steps 4-7 are SKIPPED (not failed). This prevents confusing cascade failures and clearly shows where the real problem is.

CI/CD, Docker & Environments

17.1 Jenkins Pipeline

● WHAT is it?

An automated pipeline that runs tests on every code change. Our Jenkinsfile defines: checkout code → compile → run tests → publish reports → notify on failure.

● WHY do we need it?

Manual test execution is unreliable. Someone forgets to run tests. Someone runs them against the wrong environment. CI/CD ensures tests run automatically, consistently, every time. No human error.

17.2 GitHub Actions

● WHAT is it?

GitHub's built-in CI/CD. Our workflow: push to main → smoke tests, pull request → regression tests, nightly schedule → full regression, manual trigger → any suite on any env.

● WHY do we need it?

Tests integrated into the development workflow. A PR cannot be merged until smoke tests pass. Nightly regression catches issues that accumulate over the day. Manual trigger for ad-hoc testing.

17.3 Docker

● WHAT is it?

A container that packages the framework + JDK + Maven + all dependencies into a single portable image. Run anywhere: laptop, CI server, cloud.

● WHY do we need it?

'It works on my machine' is eliminated. The Docker image is the same everywhere. JDK version, Maven version, OS libraries — all identical. No more debugging environment differences.

Step 1:

`docker build -t sbiepay-tests .` — builds the image (once).

Step 2:

`docker run --rm -e ENV=sit -e SUITE=smoke sbiepay-tests` — runs smoke on SIT.

Step 3:

Add `-v $(pwd)/reports:/app/target/reports` to get the Extent Report on your local disk.

Step 4:

In CI: the pipeline runs the Docker container instead of installing Java/Maven on the agent.

You Have the Reasoning. Now Build.

Every concept in this guide follows a pattern:

WHAT is it? → Understand the tool.

WHY do we need it? → Understand the problem it solves.

HOW does it work? → Understand the mechanics.

WHEN to use it? → Understand the context.

Apply this same WHAT/WHY/HOW/WHEN pattern to every new concept you encounter — in API testing, in system design, in life. Clear reasoning is the foundation of clear engineering.

SBlePay REST Assured Enterprise Framework • 60+ Concepts • 17
Chapters